

Phase 1: Mid-Semester Project (The MVP) - Code Use STUDENT VIEW

b38361d8-4140-4b8b-9bcd-c67ab9532f44

Attempt 1 • ⌚ Apr 5, 2026, 9:13 PM • ⬇ Apr 5, 2026, 9:14 PM

74/100

The app implements a multi-page Streamlit inventory/order manager with Manager and Customer roles, working CRUD for inventory and orders persisted to JSON, and basic authentication and registration. Core workflows (placing orders, restocking, assembling, shipping, deleting) function and write to JSON, but there are issues with inconsistent field names in orders, fragile widget keys, limited validation for registration/login, and some UI/structure polish and code organization opportunities remain.



Rubric Overview

Score distribution across all criteria

Multi-page structure and sess...		80%
Login, registration, and authe...		60%
Dual-role architecture and rol...		87%
CRUD functionality and JSO...		85%
Functionality, validation, notic...		80%
Basic layout and usability		70%
Code organization and proce...		60%
Streamlit implementation qual...		60%
Layout effort, polish, and con...		60%

Rubric Breakdown

Multi-page structure and session-state navigation

8 / 10

You implemented page branching using `st.session_state['role']` and `'page'` with clear Manager/Customer/login flows, and used `st.rerun()` after many state changes. Navigation is mostly reliable, but some branches re-read files and rerun frequently which can cause brief refresh flicker; consider centralizing page navigation and minimizing repeated rerun calls.

Login, registration, and authentication flow

9 / 15

Login and registration are present and route users into the correct role pages, and `session_state` tracks user/role. Registration lacks validation for empty or duplicate emails and stores plaintext passwords; add checks to prevent duplicate accounts and validate inputs before saving.

Dual-role architecture and role-based feature structure

13 / 15

Two distinct roles are implemented with meaningful, different features (inventory/management for Manager; browsing/ordering for Customer) and routing is clear. Make the role separation more explicit in code structure (e.g., dedicated functions per role) and ensure feature parity for common elements like order listing fields.

CRUD functionality and JSON persistence

17 / 20

Create, update, and delete actions write back to JSON files (orders and inventory) and changes persist across runs; assembly and restock update stock and save. However, inconsistent field names in Orders and multiple code paths that read/write with different keys risk data mismatches; consolidate the schema and ensure all updates modify the canonical JSON structure.

Functionality, validation, notices, and user flow

8 / 10

Main workflows work end-to-end with success/error notices and spinners, and quantity/selection validations are present in many places. Some validation is missing (registration duplicates, empty inputs in some flows), and a few error/field-handling cases (e.g., missing inventory keys) can produce crashes or unclear messages; add defensive checks and clearer user feedback.

Basic layout and usability

7 / 10

UI uses tabs, columns, containers, and sidebar which helps organization; metrics and summaries are helpful for managers. The layout is functional but inconsistent text/field presentation and some raw data table exposure (users dataframe) reduce polish; consider hiding raw dumps and aligning spacing/labels consistently.

Code organization and process separation (Part B)

3 / 5

The code is a single script with readable sections for each page, and there is some separation of concerns in blocks. Refactor into functions/modules for data access, UI rendering, and business logic to reduce duplication (many repeated file reads/writes and similar loops) and improve testability.

Streamlit implementation quality and best practices (Part B)

3 / 5

You use keys on many widgets and call `st.rerun()` after state changes which helps keep UI consistent. Nevertheless there are potential duplicate-key risks (dynamic keys vs implicit ones), and some stale UI/state issues could happen from multiple reruns and repeated file reads; tidy up keys, centralize state updates, and avoid unnecessary reruns.

Layout effort, polish, and consistency (Design Excellence — Part C)

6 / 10

The app shows a consistent basic design with sidebar and tabbed pages and some attention to metrics and containers. It remains utilitarian rather than polished: refine typography/hierarchy, hide developer/debug data, and standardize card/metric styling across pages for a more professional look.

Strengths

- Implements two distinct roles (Manager and Customer) with role-specific pages and actions.
- Performs JSON persistence for inventory and orders on updates (writing back after changes).

- Includes useful workflows: placing orders, assembling cars consuming parts, restocking, shipping, and deleting orders.

Needs Attention

- Fix inconsistent order field names (e.g., 'Order ID' vs 'Order_ID', 'Item' vs 'Item ID') so displays, updates, and reads are robust.
- Improve authentication validation and duplicate-account checks, and avoid storing plaintext passwords or exposing all users in the UI.
- Stabilize Streamlit widget key usage and reduce reliance on implicit keys to prevent duplicate widget IDs and stale UI state.